

6.0 VU Theoretische Informatik (192.017)

Teil 1: Einführung in Berechenbarkeit und Entscheidbarkeit

Reinhard Pichler und Felix Winter

Institut für Logic and Computation

Wintersemester 2026

Teil 1: Einführung in Berechenbarkeit und Entscheidbarkeit

Teil 1.1: Probleme, Programme, Algorithmen

Teil 1.2: Berechenbarkeit, Entscheidbarkeit

Teil 1.3: Semi-Entscheidbarkeit

Teil 1.4: Komplement

Teil 1.5: Jenseits der Semi-Entscheidbarkeit

Teil 1.6: Reduktionen

Probleme: Formale Definition

Definition von “Problemen”

Ein **Problem** ist definiert durch eine (abzählbar) unendliche Menge von möglichen **Instanzen** (d.h.: möglichen Inputs) zusammen mit einer Frage.

Ein **Entscheidungsproblem** ist ein Problem, bei dem die Frage eine ja/nein Antwort erwartet.

Beispiel für ein Problem

GRAPH-ERREICHBARKEIT:

INSTANZ: Ein Graph $G = (V, E)$ und Knoten $u, v \in V$.

FRAGE: Gibt es im Graph G einen Pfad von u nach v ?

GRAPH-ERREICHBARKEIT ist ein Entscheidungsproblem.

Sprechweise: Instanzen mit einer “ja”-Antwort werden als **positive** Instanzen bezeichnet; Instanzen mit einer “nein”-Antwort werden als **negative** Instanzen bezeichnet.

Einige weitere Problemtypen

d.h.: andere Fragen als die mit ja/nein Antwort

- **Funktionsproblem:** Welches Ergebnis liefert die Anwendung einer Funktion f auf eine gegebene Instanz x (d.h.: die Antwort ist $f(x)$).
- **Optimierungsproblem:** berechne den optimalen Wert (= min/max) einer Funktion $f(x)$ (abhängig von der gegebenen Instanz x), z.B.: Länge des kürzesten Pfades zwischen 2 gegebenen Knoten.

Beobachtung: In der Informatik gibt es häufig den Begriff einer “Lösung”. Die folgenden Problemtypen stellen dazu unterschiedliche Fragen.

- **Suchproblem:** Finde zu einer gegebenen Instanz *eine* Lösung, z.B.: finde einen konkreten Pfad von u nach v .
- **Aufzählproblem:** berechne alle “Lösungen” zu einer gegebenen Problem Instanz, z.B.: alle (zyklen-freien) Pfade von u nach v
- **Zählproblem:** berechne die Anzahl der “Lösungen” zu einer gegebenen Problem Instanz, z.B.: Wie viele unterschiedliche (zyklen-freie) Pfade von u nach v gibt es?

Motivation

Frage: Wieso werden in der Definition von “Problemen” *unendlich* viele Instanzen verlangt?

Antwort:

- Ein Lösungsverfahren muss auf *beliebige* Instanzen der Problemstellung anwendbar sein.
- Wenn es nur *endlich* viele Instanzen gäbe, wäre theoretisch ein Lösungsverfahren mit look-up in der Tabelle aller Instanzen samt Lösung möglich.
- *Unendlich* viele Instanzen stellen sicher, dass wir bestimmte *mathematische Werkzeuge zur Analyse der Schwierigkeit* von Problemen anwenden können.

Beispiele: Ein Lösungsverfahren für GRAPH-ERREICHBARKEIT muss auf *beliebige* Graphen anwendbar sein, eine SQL Anfrage muss für *beliebige* Daten zu gegebenem Schema gelten, etc.

Algorithmen

Definition eines Algorithmus

Ein **Algorithmus** für ein Problem $\mathcal{P}$ ist eine **Beschreibung von Rechenschritten**, die es uns erlauben, jede **beliebige** Instanz des Problems $\mathcal{P}$ zu lösen.

- Diese Definition ist ein bisschen vage ...
 - ▶ Was ist in eine zulässige “Beschreibung”?
 - ▶ Was ist ein zulässiger “Rechenschritt”?
- Natürliche Anforderung: Ein Algorithmus muss auf **alle Instanzen des Problems** anwendbar sein und die **korrekte Antwort** liefern.
- Insbesondere muss so ein Algorithmus (= Lösungsverfahren) auf jeder Instanz des Problems nach **endlich** vielen Schritten **terminieren**.
- Außerdem verlangen wir, dass
 - ③ jeder einzelne Rechenschritt **“einfach”** sein muss (d.h.: er kann von einer Maschine ausgeführt werden) und
 - ④ die Beschreibung auch für Menschen **verständlich** sein muss.

Berechenbarkeitstheorie

- **Zentrale Frage:**

Welche Probleme lassen sich mit Hilfe eines Algorithmus lösen?

- Insbesondere: Was ist, wenn wir für ein Problem keinen Algorithmus finden? Ist es unsere Schuld oder gibt es für dieses Problem gar keinen Algorithmus? Wie können wir **mit mathematischen Methoden beweisen**, dass es für ein Problem keinen Algorithmus gibt?
- Gibt es überhaupt Probleme, für die kein Algorithmus existiert?
- Um mathematische Methoden anwenden zu können, brauchen wir ein **Modell** für den Berechenbarkeitsbegriff.
- **Berechenbarkeitsmodell** im ersten Teil der Vorlesung: eine einfache imperative Programmiersprache (einfache Datentypen, Variablen, Wertzuweisung, if-then-else, while, etc.)
- Im zweiten Teil der Vorlesung: einfachere Modelle, mit denen sich auch kompliziertere formale Beweise führen lassen.

Programme

Programmiersprachen

- Programmiersprachen erlauben uns, **Programme** zu schreiben, die eine **formale** Beschreibung von Rechenschritten darstellen.
- Programme sind für Menschen verständlich / nachvollziehbar und können von Computern ausgeführt werden.

Berechenbarkeit

- Wir wählen eine Programmiersprache als unser **Modell für Berechenbarkeit**.
- Damit können wir eine zentrale Frage beantworten: Gibt es für ein gegebenes Problem ein Lösungsverfahren (= einen **Algorithmus**)?

Unser Programmiersprache: SIMPLE

- Wir verwenden den Kern von prozeduralen Programmiersprachen:
 - ▶ **Variablen** von unterschiedlichem Typ (String, Integer, Boolean, Void)
 - ▶ **Wertzuweisungen, einfache Rechenoperationen** (z.B.: $x := y + z$)
 - ▶ **“if/then/else”** Anweisungen
 - ▶ **“while”** Schleifen
 - ▶ **“for”** und **“repeat”** Schleifen
(können durch “while” Schleifen ersetzt werden)
 - ▶ **“return”** Anweisungen
 - ▶ **Prozeduren, Funktionen**
- Der **Input** für ein Programm kann sein:
 - ▶ eine Liste $L = (V_1, V_2, \dots, V_n)$ von Werten unterschiedlicher Typen;
 - ▶ oder einfach ein (großer) String I : Jede Liste L lässt sich als String über einem beliebigen “Alphabet” darstellen (z.B.: ASCII, $\{0, 1\}$, etc.)
- Ein Programm liefert als Ergebnis einen Wert (von beliebigem Typ) mittels “return” Anweisung.
- Es ist nicht garantiert, dass ein Programm auf einem gegebenen Input terminiert. Die vom Programm berechnete Funktion ist dann auf diesem Input nicht definiert, d.h.: wir haben eine **partielle Funktion**.

Unsere Programmiersprache (Beispiel)

Problem: Primzahltest

INSTANZ: Natürliche Zahl n .

FRAGE: Ist n eine Primzahl?

Programm (Algorithmus)

Boolean isPrime(Integer n)

if $n < 2$ **then return** false;

$i := 2$;

while $i < n$ **do** {

if $n \% i = 0$ **then return** false;

$i := i + 1$; }

return true;

Angemessenheit unserer Programmiersprache SIMPLE

- Ist die Programmiersprache SIMPLE ein **geeignetes Modell** für Berechenbarkeit?
 - ▶ d.h.: **Lassen sich in SIMPLE alle Algorithmen ausdrücken?**
 - ▶ Oder gibt es Probleme, für die es zwar ein Programm in Java, C oder einer anderen Programmiersprache aber kein SIMPLE Programm gibt?
- Antwort: **SIMPLE ist ein geeignetes Modell**, d.h. alle Probleme, die in JAVA, C oder einer anderen bekannten Programmiersprache gelöst werden können, lassen sich auch in SIMPLE lösen.
- Begründung:
 - ▶ In SIMPLE lassen sich **Interpreter** oder **Compiler** für alle bekannten Programmiersprachen implementieren, z.B.:
 - ▶ eine Java Virtual Machine (JVM), um JAVA Programme auszuführen.
- Konsequenz: Wenn wir beweisen können, dass es für ein Problem kein SIMPLE Programm gibt, dann gibt es generell keinen Algorithmus für dieses Problem (auch nicht in einer anderen Programmiersprache).

Teil 1: Einführung in Berechenbarkeit und Entscheidbarkeit

Teil 1.1: Probleme, Programme, Algorithmen

Teil 1.2: Berechenbarkeit, Entscheidbarkeit

Teil 1.3: Semi-Entscheidbarkeit

Teil 1.4: Komplement

Teil 1.5: Jenseits der Semi-Entscheidbarkeit

Teil 1.6: Reduktionen

Berechenbarkeit

Berechenbarkeit von Funktionen

Wir nennen eine Funktion f **berechenbar**, wenn es einen Algorithmus gibt, der ein beliebiges Element x der Grundmenge von f als Input nimmt und als Output den Funktionswert $f(x)$ liefert.

Bemerkung:

- Im Prinzip kann die Grundmenge aus beliebigen Objekten bestehen, z.B.: Zahlen, Strings, Graphen, Datenbanken (möglicherweise eingeschränkt durch ein Schema), etc.
- Die Objekte müssen sich als (endliche) Strings über einem geeigneten (endlichen) Alphabet codieren lassen.
- wichtige Arten von hier betrachteten Funktionen:
 $f: \mathbb{N} \rightarrow \mathbb{N}$, $f: \Sigma^* \rightarrow \Sigma^*$, $f: \mathbb{N} \rightarrow \{0, 1\}$, $f: \Sigma^* \rightarrow \{0, 1\}$.
- Statt $\{0, 1\}$ kann man auch $\{\text{nein}, \text{ja}\}$ oder $\{\text{falsch}, \text{wahr}\}$ nehmen, d.h.: das *Funktionsproblem* ist eigentlich ein *Entscheidungsproblem*.

Entscheidbarkeit

Entscheidbarkeit

Wir nennen ein Entscheidungsproblem $\mathcal{P}$ **entscheidbar**, wenn es einen Algorithmus gibt, der **eine beliebige Instanz** x des Problems $\mathcal{P}$ als Input nimmt und die **korrekte Antwort** ja / nein (oder 1 / 0 oder wahr / falsch) als Output liefert (insbesondere **terminiert** so ein Algorithmus auf jeder Instanz).

Bemerkung:

- Unser Modell für Berechenbarkeit sind SIMPLE Programme.
- d.h.: Eine Funktion ist **berechenbar** bzw. ein Entscheidungsproblem ist **entscheidbar**, wenn es dafür ein **SIMPLE Programm** gibt.
- offensichtlich: viele Funktionen sind berechenbar und viele Entscheidungsprobleme sind entscheidbar (siehe z.B.: AlgoDat).
- weniger offensichtlich: es gibt auch nicht-berechenbare Funktionen und nicht-entscheidbare Entscheidungsprobleme.

Partielle vs. Totale Funktionen

Berechenbare Funktionen

In der **Berechenbarkeitstheorie** wird eine (möglicherweise partielle) Funktion $f: X \rightarrow Y$ als **berechenbar** bezeichnet, wenn es einen Algorithmus A mit folgender Eigenschaft gibt:

- Algorithmus A nimmt ein beliebiges Element $x \in X$ als Input;
- Wenn A auf Input x terminiert, dann liefert er als Output $f(x)$;
- Algorithmus A läuft auf Input x **endlos** $\Leftrightarrow f$ ist auf x **nicht definiert**.

Bemerkungen.

- Für eine **berechenbare, totale** Funktion gilt also: es gibt einen Algorithmus A , der auf *jedem* Element $x \in X$ *terminiert* und den korrekten Wert $f(x)$ als Output liefert.
- Von einem **Lösungsverfahren** für eines der am Beginn von Teil 1 der Vorlesung genannten Probleme (Entscheidungs-, Optimierungs-, Aufzähl-, Zählprobleme) verlangen wir, dass es auf jeder Instanz terminiert und die korrekte Antwort liefert.

Existenz von unentscheidbaren Problemen

Beobachtung

- SIMPLE Programme sind endliche Strings über einem endlichen Alphabet Σ (z.B. 127 ASCII-Zeichen).
- Auch die Instanzen eines Entscheidungsproblems können als Strings über Σ dargestellt werden.
- Entscheidungsprobleme entsprechen dann Funktionen $\Sigma^* \rightarrow \{0, 1\}$.

Theorem

- Die Menge Σ^* ist *abzählbar unendlich*, d.h.: es existiert eine bijektive Funktion $g : \Sigma^* \rightarrow \mathbb{N}$.
 $g: \mathbb{N} \rightarrow \{0, 1\}$
- Die Menge der Funktionen $\mathbb{N} \rightarrow \{0, 1\}$ ist *überabzählbar* (und daher ist auch die Menge der Funktionen $\Sigma^* \rightarrow \{0, 1\}$ überabzählbar).

Existenz von unentscheidbaren Problemen

Zusammenfassung

- Es gibt also überabzählbar viele Funktionen und Entscheidungsprobleme.
- Es gibt aber nur abzählbar viele endliche Strings über einem endlichen Alphabet. Daher gibt es nur abzählbar viele SIMPLE Programme.
- d.h.: es muss Funktionen und Entscheidungsprobleme geben, für die es kein SIMPLE Programm (und somit keinen Algorithmus) gibt.

Was wir noch zeigen müssen:

- Abzählbarkeit von Σ^*
- Überabzählbarkeit von $\{0, 1\}^{\mathbb{N}}$
(d.h.: Menge der Funktionen $f : \mathbb{N} \rightarrow \{0, 1\}$)

Abzählbarkeit von Σ^*

Beweisidee:

Angenommen $\Sigma = \{a_1, \dots, a_k\}$. Dann können wir die Menge Σ^* auf folgende Weise **aufzählen**:

- Aufzählung der Strings geordnet nach der Länge
- Anordnung innerhalb der selben Länge: lexikographisch

Eine bijektive Abbildung $g: \Sigma^* \rightarrow \mathbb{N}$ erhält man, indem man jedes **Element** in der Aufzählung auf seine **Position** in der Aufzählung abbildet.

Aufzählung:

Strings	Anzahl	Positionen
ϵ (leerer String)	1	0
$a_1, \dots, a_k$	k	$1, \dots, k$
$a_1 a_1, a_1 a_2, \dots, a_k a_k$	k^2	$k + 1, \dots, k^2 + k$
$a_1 a_1 a_1, a_1 a_1 a_2, \dots, a_k a_k a_k$	k^3	$k^2 + k + 1, \dots, k^3 + k^2 + k$
etc.		

Überabzählbarkeit von $\{0, 1\}^{\mathbb{N}}$

Beweisidee (Cantor'sches Diagonalverfahren) :

- Indirekt: angenommen, die Funktionen in $\{0, 1\}^{\mathbb{N}}$ sind abzählbar. Dann lassen sie sich (so wie die Elemente in Σ^*) aufzählen.
- Jede Funktion lässt sich als Folge von Nullen und Einsen darstellen.
- Indem wir entlang der “Diagonale” jeweils den Wert “umdrehen”, erzeugen wir eine Zahl, die in der Aufzählung nicht vorkommt.

Aufzählung von $\{0, 1\}^{\mathbb{N}}$:

f_0	=	a₀₀	a_{01}	a_{02}	a_{03}	a_{04}	...
f_1	=	a_{10}	a₁₁	a_{12}	a_{13}	a_{14}	...
f_2	=	a_{20}	a_{21}	a₂₂	a_{23}	a_{24}	...
$\vdots$	=	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	

Definiere nun eine Funktion $\hat{f}: \mathbb{N} \rightarrow \{0, 1\}$ mit $\hat{f}(n) = 1 - f_n(n)$.

Diese Funktion ist in der Aufzählung nicht enthalten. Widerspruch!

Bemerkung: Georg Cantor hat so die Überabzählbarkeit von $\mathbb{R}$ gezeigt.

Probleme über Programme

- Es gibt viele natürliche Probleme, für die ein Algorithmus nicht offensichtlich ist! Dazu gehören insbesondere zahlreiche Probleme, die Fragen zum **Verhalten von Programmen** betreffen, z.B.:
 - ▶ Gegeben ein Programm Π und ein Input I , tritt das Programm irgendwann in eine Endlosschleife?
 - ▶ Gegeben ein Programm Π , terminiert es auf *allen* Inputs?
- Algorithmen für solche Probleme wären sehr hilfreich, um die Korrektheit von Programmen sicherzustellen.
- Außerdem könnten einige (zum Teil “klassische”) mathematische Probleme gelöst werden, wenn wir einen Algorithmus zum Entscheiden der Programm-Termination hätten, z.B.:
 - ▶ Goldbachsche Vermutung, die eines der berühmtesten offenen Probleme der Mathematik darstellt (aufgestellt 1742 von Christian Goldbach in einem Brief an Leonhard Euler).

Goldbachsche Vermutung

Jede gerade Zahl größer als 2 ist die Summe von 2 Primzahlen.

Wir können ein Programm schreiben, das diese Aussage für 4, 6, 8, 10, 12, ... überprüft.

```
Boolean test(Integer n)  /* checks if n is the sum of two primes */  
    for all  $i \leq n, j \leq n$  do {  
        if (isPrime(i) and isPrime(j) and  $i + j = n$ ) then return true; }  
    return false;
```

```
Void testConjecture()  
    n:=4;  
    while test(n)=true do {  $n := n + 2$ ; }
```

Theorem

Die Goldbachsche Vermutung ist wahr $\Leftrightarrow$ testConjecture() terminiert nicht.

Versuch eines computergestützten Beweises

Idee

- Angenommen wir hätten ein Programm Π_h , das **Programm-Termination** entscheidet, d.h.: Π_h nimmt als Input ein Programm Π (d.h.: dessen Quellcode) und möglicherweise einen Input String I für Π und entscheidet, ob das Programm Π auf diesem Input terminiert.
- Dann könnten wir Π_h auf das Programm `testConjecture()` anwenden und entscheiden, ob `testConjecture()` terminiert.
- Damit könnten wir die Gültigkeit der Goldbachschen Vermutung entscheiden.
 - ▶ Falls Π_h "nein" ausgibt, ist die Vermutung bewiesen.
 - ▶ Falls Π_h "ja" ausgibt, ist die Vermutung widerlegt, d.h., es gibt eine gerade Zahl $n > 2$, die nicht die Summe von 2 Primzahlen ist.

Bemerkung: Zahlreiche andere mathematische Probleme könnten mittels Algorithmus zum Entscheiden der Programm-Termination gelöst werden, z.B.: der große Fermatsche Satz.

Unentscheidbarkeit des HALTEPROBLEMS

HALTEPROBLEM

INSTANZ: (Quellcode von einem) Programm Π , Input String I .

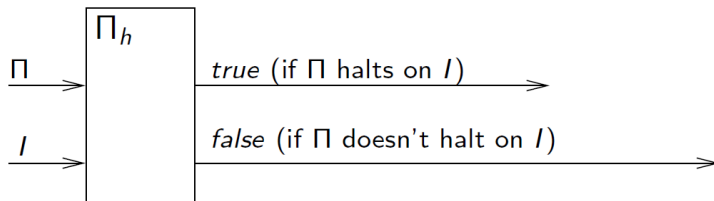
FRAGE: Terminiert das Programm Π auf dem Input I ?

- Wir beweisen, dass es keinen Algorithmus (also kein Programm Π_h) gibt, das das HALTEPROBLEM entscheidet.
- Der Beweis ist **indirekt**, d.h.: wir nehmen an, dass das HALTEPROBLEM mit Hilfe eines Programms Π_h entscheidbar ist und zeigen, dass diese Annahme zu einem **Widerspruch** führt.
- Hier (und bei allen weiteren Problemen über Programme) nehmen wir an, dass der Quellcode des Programms Π als String gegeben ist und somit selbst als Input für ein Programm verwendet werden kann.
- “**Diagonalargument**”: Wir werden letztlich ein Programm “auf sich selbst” (sprich: auf seinem Quellcode) laufen lassen, z.B.: $\Pi(\Pi)$ bedeutet, dass das Programm Π auf dem Input Π (sprich: auf dem Quellcode des Programms) läuft.

Unentscheidbarkeit des HALTEPROBLEMS (Schritt 1)

Wie nehmen an, dass es ein Programm Π_h mit folgender Eigenschaft gibt:

- Π_h nimmt 2 Strings als Input:
 - ▶ Π (Quellcode eines SIMPLE Programms)
 - ▶ I (Input für das Programm Π)
- Π_h erzeugt folgenden Output:
 - ▶ *true* falls das Programm Π auf Input I terminiert;
 - ▶ *false* falls das Programm Π auf Input I *nicht* terminiert.

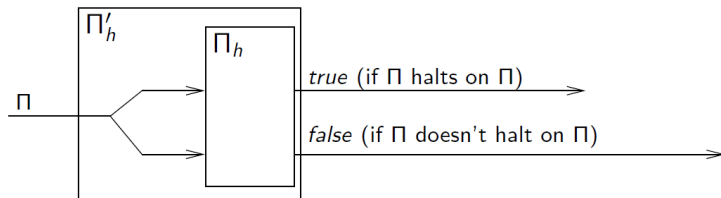


```
Boolean  $\Pi_h$  (String  $\Pi$ , String  $I$ )  
    /* code to check if  $\Pi$  terminates on  $I$ . */
```

Unentscheidbarkeit des HALTEPROBLEMS (Schritt 2)

Mit Hilfe von Π_h erzeugen wir nun ein Programm Π'_h :

- Π'_h nimmt als Input *einen* String, dupliziert diesen und ruft mit diesen 2 Strings das Programm Π_h auf.
- d.h.: Π'_h prüft, ob ein Program Π terminiert, wenn es seinen eigenen Quellcode als Input nimmt.

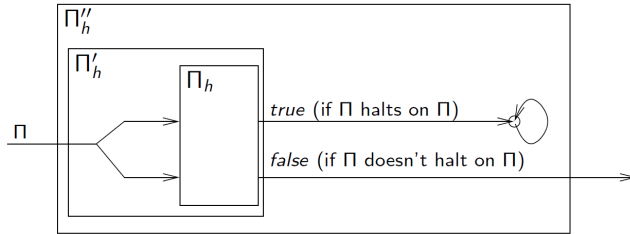


```
Boolean  $\Pi'_h$  (String  $\Pi$ )  
    return  $\Pi_h(\Pi, \Pi)$ ;
```

Unentscheidbarkeit des HALTEPROBLEMS (Schritt 3)

Mit Hilfe von Π'_h erzeugen wir nun ein weiteres Programm Π''_h (und zwar verändern wir das Output-Verhalten von Π'_h):

- Falls Π'_h *true* liefert, dann geht Π''_h in eine Endlosschleife!



Boolean Π''_h (String Π)

```
if  $\Pi'_h(\Pi) = \text{true}$  then { while (true) do { } }  
    else return false;
```

Unentscheidbarkeit des HALTEPROBLEMS (Schritt 4)

Beobachtung:

- ❶ Falls Π auf dem Input Π hält, dann hält Π''_h auf dem Input Π *nicht*.
- ❷ Falls Π auf dem Input Π *nicht* hält, dann hält Π''_h auf dem Input Π .

Was passiert, wenn Π''_h mit (dem Quellcode von) Π''_h als Input läuft?

Es kann nur zwei Möglichkeiten geben:

- ❶ Fall 1: Π''_h hält auf dem Input Π''_h . Dann folgt aus Beobachtung 1, dass Π''_h *nicht* auf dem Input Π''_h hält. **Widerspruch!**
- ❷ Fall 2: Π''_h hält *nicht* auf dem Input Π''_h . Dann folgt aus Beobachtung 2, dass Π''_h auf dem Input Π''_h hält. **Widerspruch!**

d.h.: unsere ursprüngliche Annahme (nämlich, dass es ein Programm Π_h gibt, das das HALTEPROBLEM entscheidet) war falsch und es gilt:

Theorem

Das HALTEPROBLEM ist unentscheidbar.

Weitere Beispiele von unentscheidbaren Problemen

KORREKTHEIT

INSTANZ: (Quellcode von einem) Programm Π und 2 Strings I_1, I_2 .

FRAGE: Terminiert Π auf dem Input I_1 und liefert es als Output I_2 ?

Intuition: KORREKTHEIT ist unentscheidbar, weil die Beantwortung dieser Frage implizit die Frage beantworten müsste, ob Π auf Input I_1 hält.

ERREICHBARER-CODE

INSTANZ: (Quellcode von einem) Programm Π , ein Label (= String) L .

FRAGE: Gibt es einen Input I , so dass das Programm Π bei Ausführung mit Input I den Code auf der Programmzeile mit dem Label L ausführt?

Optimierungspotenzial: unerreichbarer Code kann gelöscht werden.

Intuition: ERREICHBARER-CODE ist unentscheidbar, weil wir “eine Art HALTEPROBLEM” erhalten, wenn die *letzte* Zeile in Π Label L bekommt.

Bemerkung: Diese intuitiven Argumente für Unentscheidbarkeit werden wir formalisieren, wenn wir den Begriff der “**Reduktion**” einführen.

Teil 1: Einführung in Berechenbarkeit und Entscheidbarkeit

Teil 1.1: Probleme, Programme, Algorithmen

Teil 1.2: Berechenbarkeit, Entscheidbarkeit

Teil 1.3: Semi-Entscheidbarkeit

Teil 1.4: Komplement

Teil 1.5: Jenseits der Semi-Entscheidbarkeit

Teil 1.6: Reduktionen

Semi-entscheidbare Probleme

Wir lockern die Bedingung für Entscheidbarkeit und definieren semi-entscheidbare Probleme:

Definition

Wir nennen ein Entscheidungsproblem $\mathcal{P}$ **semi-entscheidbar**, wenn es ein Programm Π mit folgenden Eigenschaften gibt:

- Π nimmt eine Instanz I von $\mathcal{P}$ als Input;
- Falls I eine “ja” Instanz ist, dann liefert Π das Ergebnis *true*;
- falls I eine “nein” Instanz ist, dann kann Π entweder das Ergebnis *false* liefern oder nicht terminieren.

In anderen Worten:

- Π arbeitet korrekt auf allen positiven Instanzen von $\mathcal{P}$;
- Π darf auf negativen Instanzen von $\mathcal{P}$ endlos laufen;
- aber wenn Π auf einer negativen Instanz terminiert, dann muss Π das korrekte Ergebnis (= *false*) liefern.

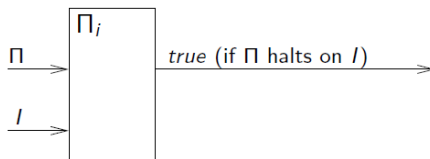
Noch einmal das HALTEPROBLEM

Theorem

Das HALTEPROBLEM ist semi-entscheidbar.

Beweisidee: Wir können ein **Interpreter** Programm Π_i mit folgenden Eigenschaften schreiben:

- Π_i nimmt als Input eine beliebige Instanz des HALTEPROBLEMS, d.h.: Quellcode von einem Programm Π und Input I für Π ;
- Π_i analysiert Π und simuliert die Ausführung von Π auf Input I ;
- Wenn die Simulation von Π auf I **terminiert**, gibt Π_i **true** aus und terminiert;
- Wenn die Simulation von Π auf I **nicht terminiert**, dann läuft auch Π_i auf dem Input (Π, I) **endlos**.



Weitere semi-entscheidbare Probleme

Wiederholung: KORREKTHEIT

INSTANZ: (Quellcode von einem) Programm Π und 2 Strings I_1, I_2 .

FRAGE: Liefert Π bei Aufruf mit Input I_1 als Output den String I_2 ?

Theorem

Das KORREKTHEIT-Problem ist semi-entscheidbar.

Beweisidee (analog zum HALTEPROBLEM): Wir können ein **Interpreter** Programm Π_i mit folgenden Eigenschaften schreiben:

- Π_i nimmt als Input eine beliebige Instanz vom KORREKTHEIT d.h.: Quellcode von einem Programm Π und zwei Strings I_1, I_2 ;
- Π_i analysiert Π und simuliert die Ausführung von Π auf Input I_1 ;
- Wenn die Simulation **mit Output O terminiert**, dann überprüft Π_i , ob $I_2 = O$ gilt. Wenn ja, dann gibt Π_i **true** aus, sonst **false**;
- Wenn die Simulation von Π auf I_1 **nicht terminiert**, dann läuft auch Π_i auf dem Input (Π, I) **endlos**.

Weitere semi-entscheidbare Probleme

Wiederholung: ERREICHBARER-CODE

INSTANZ: (Quellcode von einem) Programm Π , ein Label (= String) L .

FRAGE: Gibt es einen Input I , so dass das Programm Π bei Ausführung mit Input I den Code auf der Programmzeile mit dem Label L ausführt?

Beobachtungen: Für ein gegebenes Paar (I, i) , wobei I ein möglicher Input für Π ist und $i \in \mathbb{N}$, können wir

- einen Interpreter ausführen, der die ersten i Schritte von Π auf Input I simuliert, und
- der entscheidet, ob innerhalb der ersten i Schritte die Zeile mit Label L im Programm Π erreicht wurde.

Idee des Semi-Entscheidungsverfahrens:

- Alle Paare (I, i) aufzählen und überprüfen, ob Π bei Input I die Programmzeile mit Label L innerhalb von i Schritten erreicht.
- Falls (Π, L) eine positive Instanz ist, werden wir *garantiert irgendwann* so ein Paar (I, i) finden und geben *true* aus.
- Falls (Π, L) eine negative Instanz ist, läuft die Aufzählung endlos.

Einschub: Aufzählung, Abzählbarkeit

Frage: Wie können wir in einem Semi-Entscheidungsverfahren für das ERREICHBARER-CODE-Problem **alle** Paare (I, i) aufzählen?

Vorsicht: Eine **geschachtelte Schleife funktioniert nicht**, z.B.:

for all input $I \in \Sigma^*$

for all $i \in \mathbb{N}$

do simulierte die ersten i Programmschritte von Π auf Input I ;

Begründung: Wenn für einen bestimmten Input I das Programm Π nicht terminiert, werden nach I keine weiteren Inputs mehr erzeugt, d.h.: wir testen also **nicht alle** Paare (I, i) .

Allgemeine Problemstellung:

- Wie kann man $M_1 \times M_2$ aufzählen für zwei *abzählbar unendliche* Mengen M_1, M_2 ?
- Äquivalente Frage: Ist $M_1 \times M_2$ abzählbar, wenn die zwei Mengen M_1, M_2 *abzählbar unendlich* sind?

Abzählbarkeit (Fortsetzung)

Definition

Eine Menge M heißt **abzählbar**, wenn sie entweder endlich ist oder wenn es eine bijektive Abbildung $M \rightarrow \mathbb{N}$ gibt.

Beobachtungen:

- Falls M unendlich ist, müssen wir also **eine bijektive Abbildung** $f: M \rightarrow \mathbb{N}$ finden, um die Abzählbarkeit von M zu beweisen.
- Einfacher: definiere bijektive Abbildung $g: \mathbb{N} \rightarrow M$ als **Aufzählung**.
- **Alternative**: es reicht, eine **injektive Abbildung** $f: M \rightarrow \mathbb{N}$ zu finden.
- **Intuition**: Wenn es eine injektive Abbildung $f: M \rightarrow \mathbb{N}$ gibt, dann gilt offensichtlich $|M| \leq |\mathbb{N}|$.
- **Formale Begründung**: Für eine injektive Abbildung $f: M \rightarrow \mathbb{N}$ definieren wir folgende Abbildung g :

$$g: M \rightarrow \mathbb{N} \text{ mit } g(a) = |\{m \in M \mid f(m) < f(a)\}| \text{ für alle } a \in M$$

Man sieht leicht, dass g bijektiv (d.h. injektiv und surjektiv) ist.

Einige abzählbar unendliche Mengen

- $M = \mathbb{N} \cup \{a\}$ für beliebiges Objekt a :
 $f: M \rightarrow \mathbb{N}$ mit $f(a) = 0$ und $f(n) = n + 1$ für alle $n \in \mathbb{N}$ ist injektiv.
- $M = \mathbb{N} \cup \{a_1, \dots, a_k\}$ für $k \geq 1$ und beliebige Objekte $a_1, \dots, a_k$:
 $f: M \rightarrow \mathbb{N}$ mit $f(a_i) = i - 1$ und $f(n) = n + k$ für alle $n \in \mathbb{N}$
- $M = \{0, 1\} \times \mathbb{N}$:
 $f: M \rightarrow \mathbb{N}$ mit $f(i, n) = 2n + i$ für alle $i \in \{0, 1\}$ und $n \in \mathbb{N}$.
- $M = \mathbb{N} \times \mathbb{N}$ oder, allgemein:
 $M = M_1 \times M_2$ für abzählbar unendliche Mengen M_1, M_2 .

Idee: für $M = \mathbb{N} \times \mathbb{N}$:

Aufzählung (Cantor'sches Abzählprinzip):

$(0,0)$,

$(1,0)$, $(1,1)$, $(0,1)$,

$(2,0)$, $(2,1)$, $(2,2)$, $(1,2)$, $(0,2)$

$(3,0)$, $(3,1)$, $(3,2)$, $(3,3)$, $(2,3)$, $(1,3)$, $(0,3)$,

etc.

Cantor'sches Abzählprinzip

Aufzählung von $M_1 \times M_2$ mit $M_1 = \{a_1, a_2, a_3, a_4, \dots\}$ und $M_2 = \{b_1, b_2, b_3, b_4, \dots\}$:

$$\left(\begin{array}{cccc} (a_1, b_1) & (a_1, b_2) & (a_1, b_3) & \dots, \\ (a_2, b_1) & (a_2, b_2) & (a_2, b_3) & \dots, \\ (a_3, b_1) & (a_3, b_2) & (a_3, b_3) & \dots, \\ (a_4, b_1) & (a_4, b_2) & (a_4, b_3) & \dots, \\ \vdots & \vdots & \vdots & \end{array} \right) \quad \left(\begin{array}{ccccc} 1 & 4 & 9 & 16 & \dots, \\ 2 & 3 & 8 & 15 & \dots, \\ 5 & 6 & 7 & 14 & \dots, \\ 10 & 11 & 12 & 13 & \dots, \\ \vdots & \vdots & \vdots & \vdots & \end{array} \right)$$

Cantor'sches Abzählprinzip

Aufzählung von $M_1 \times M_2$ mit $M_1 = \{a_1, a_2, a_3, a_4, \dots\}$ und $M_2 = \{b_1, b_2, b_3, b_4, \dots\}$:

$$\left(\begin{array}{cccc} (a_1, b_1) & (a_1, b_2) & (a_1, b_3) & \dots, \\ (a_2, b_1) & (a_2, b_2) & (a_2, b_3) & \dots, \\ (a_3, b_1) & (a_3, b_2) & (a_3, b_3) & \dots, \\ (a_4, b_1) & (a_4, b_2) & (a_4, b_3) & \dots, \\ \vdots & \vdots & \vdots & \end{array} \right) \quad \left(\begin{array}{ccccc} 1 & 4 & 9 & 16 & \dots, \\ 2 & 3 & 8 & 15 & \dots, \\ 5 & 6 & 7 & 14 & \dots, \\ 10 & 11 & 12 & 13 & \dots, \\ \vdots & \vdots & \vdots & \vdots & \end{array} \right)$$

Cantor'sches Abzählprinzip

Aufzählung von $M_1 \times M_2$ mit $M_1 = \{a_1, a_2, a_3, a_4, \dots\}$ und $M_2 = \{b_1, b_2, b_3, b_4, \dots\}$:

$$\left(\begin{array}{cccc} (a_1, b_1) & (a_1, b_2) & (a_1, b_3) & \dots, \\ (a_2, b_1) & (a_2, b_2) & (a_2, b_3) & \dots, \\ (a_3, b_1) & (a_3, b_2) & (a_3, b_3) & \dots, \\ (a_4, b_1) & (a_4, b_2) & (a_4, b_3) & \dots, \\ \vdots & \vdots & \vdots & \end{array} \right) \quad \left(\begin{array}{ccccc} 1 & 4 & 9 & 16 & \dots, \\ 2 & 3 & 8 & 15 & \dots, \\ 5 & 6 & 7 & 14 & \dots, \\ 10 & 11 & 12 & 13 & \dots, \\ \vdots & \vdots & \vdots & \vdots & \end{array} \right)$$

Cantor'sches Abzählprinzip

Aufzählung von $M_1 \times M_2$ mit $M_1 = \{a_1, a_2, a_3, a_4, \dots\}$ und $M_2 = \{b_1, b_2, b_3, b_4, \dots\}$:

$$\left(\begin{array}{cccc} (a_1, b_1) & (a_1, b_2) & (a_1, b_3) & \dots, \\ (a_2, b_1) & (a_2, b_2) & (a_2, b_3) & \dots, \\ (a_3, b_1) & (a_3, b_2) & (a_3, b_3) & \dots, \\ (a_4, b_1) & (a_4, b_2) & (a_4, b_3) & \dots, \\ \vdots & \vdots & \vdots & \end{array} \right) \quad \left(\begin{array}{ccccc} 1 & 4 & 9 & 16 & \dots, \\ 2 & 3 & 8 & 15 & \dots, \\ 5 & 6 & 7 & 14 & \dots, \\ 10 & 11 & 12 & 13 & \dots, \\ \vdots & \vdots & \vdots & \vdots & \end{array} \right)$$

Cantor'sches Abzählprinzip

Aufzählung von $M_1 \times M_2$ mit $M_1 = \{a_1, a_2, a_3, a_4, \dots\}$ und $M_2 = \{b_1, b_2, b_3, b_4, \dots\}$:

$$\left(\begin{array}{cccc} (a_1, b_1) & (a_1, b_2) & (a_1, b_3) & \dots, \\ (a_2, b_1) & (a_2, b_2) & (a_2, b_3) & \dots, \\ (a_3, b_1) & (a_3, b_2) & (a_3, b_3) & \dots, \\ (a_4, b_1) & (a_4, b_2) & (a_4, b_3) & \dots, \\ \vdots & \vdots & \vdots & \end{array} \right) \quad \left(\begin{array}{ccccc} 1 & 4 & 9 & 16 & \dots, \\ 2 & 3 & 8 & 15 & \dots, \\ 5 & 6 & 7 & 14 & \dots, \\ 10 & 11 & 12 & 13 & \dots, \\ \vdots & \vdots & \vdots & \vdots & \end{array} \right)$$

Cantor'sches Abzählprinzip

Aufzählung von $M_1 \times M_2$ mit $M_1 = \{a_1, a_2, a_3, a_4, \dots\}$ und $M_2 = \{b_1, b_2, b_3, b_4, \dots\}$:

$$\left(\begin{array}{cccc} (a_1, b_1) & (a_1, b_2) & (a_1, b_3) & \dots, \\ (a_2, b_1) & (a_2, b_2) & (a_2, b_3) & \dots, \\ (a_3, b_1) & (a_3, b_2) & (a_3, b_3) & \dots, \\ (a_4, b_1) & (a_4, b_2) & (a_4, b_3) & \dots, \\ \vdots & \vdots & \vdots & \end{array} \right) \quad \left(\begin{array}{ccccc} 1 & 4 & 9 & 16 & \dots, \\ 2 & 3 & 8 & 15 & \dots, \\ 5 & 6 & 7 & 14 & \dots, \\ 10 & 11 & 12 & 13 & \dots, \\ \vdots & \vdots & \vdots & \vdots & \end{array} \right)$$

Daraus folgt:

Theorem

Das ERREICHBARER-CODE Problem ist semi-entscheidbar.

Cantor'sches Abzählprinzip

Aufzählung von $M_1 \times M_2$ mit $M_1 = \{a_1, a_2, a_3, a_4, \dots\}$ und $M_2 = \{b_1, b_2, b_3, b_4, \dots\}$:

$$\left(\begin{array}{cccc} (a_1, b_1) & (a_1, b_2) & (a_1, b_3) & \dots, \\ (a_2, b_1) & (a_2, b_2) & (a_2, b_3) & \dots, \\ (a_3, b_1) & (a_3, b_2) & (a_3, b_3) & \dots, \\ (a_4, b_1) & (a_4, b_2) & (a_4, b_3) & \dots, \\ \vdots & \vdots & \vdots & \end{array} \right) \quad \left(\begin{array}{ccccc} 1 & 4 & 9 & 16 & \dots, \\ 2 & 3 & 8 & 15 & \dots, \\ 5 & 6 & 7 & 14 & \dots, \\ 10 & 11 & 12 & 13 & \dots, \\ \vdots & \vdots & \vdots & \vdots & \end{array} \right)$$

Daraus folgt:

Theorem

Das ERREICHBARER-CODE Problem ist semi-entscheidbar.

Bemerkung: Georg Cantor hat so die Abzählbarkeit von $\mathbb{Q}$ gezeigt.

Weitere semi-entscheidbare Probleme

Das “ENTSCHEIDUNGSPROBLEM”

INSTANZ: eine Formel ϕ der Prädikatenlogik erster Stufe.

FRAGE: Ist ϕ gültig?

Theorem

Das ENTSCHEIDUNGSPROBLEM ist semi-entscheidbar.

Wiederholung: Prädikatenlogische Formeln erster Stufe sind so definiert, dass man zuerst induktiv **Terme** definiert und damit dann ebenfalls induktiv **Formeln**:

Terme können sein:

- Konstantensymbole (üblicherweise $a, b, c, \dots$)
- Variablen (üblicherweise x, y, z)
- zusammengesetzte Terme der Form $f(t_1, \dots, t_\alpha)$ für ein Funktionssymbol f mit Arität α und Terme $t_1, \dots, t_\alpha$.

Das ENTSCHEIDUNGSPROBLEM

Formeln können sein:

- Atome der Form $P(t_1, \dots, t_\alpha)$ für ein Prädikatensymbol P mit Arität α und Terme $t_1, \dots, t_\alpha$
- oder zusammengesetzte Formeln der Form $\neg\phi$, $\phi \wedge \psi$, $\phi \vee \psi$, $(\exists x)\phi$ oder $(\forall x)\phi$ für Formeln ϕ, ψ und Variable x .

Geschichte:

- Das Entscheidungsproblem (der Prädikatenlogik erster Stufe) wurde bereits von Gottfried Leibniz im 17. Jahrhundert formuliert.
- Anfang des 20. Jahrhunderts von David Hilbert und Wilhelm Ackermann neu auf die “Tagesordnung” der Mathematik gesetzt.
- Semi-Entscheidbarkeit: aufgrund des “Vollständigkeitssatzes” von Kurt Gödel.
- Unentscheidbarkeit: unabhängig voneinander von Alonzo Church 1935 (mit λ -calculus als Modell für Berechenbarkeit) und Alan Turing 1936 (mit Turing Maschinen als Modell für Berechenbarkeit) bewiesen.

Das ENTSCHEIDUNGSPROBLEM

Idee eines Unentscheidbarkeitsbeweises:

- Mittels **Reduktion** vom **HALTEPROBLEM** für **Turing Maschinen**.
- Reduktionen werden noch im Lauf von Teil 1 der LVA eingeführt; Turing Maschinen im Teil 2.

Idee eines Semi-Entscheidungsverfahrens:

- Es gibt **vollständige Kalküle** für die Prädikatenlogik erster Stufe, d.h.: in einem vollständigen Kalkül gibt es für jede gültige Formel eine **endliche Ableitung**.
- z.B.: **Hilbert Kalkül**:
 - ▶ einige **Axiome** wie $A \rightarrow (B \rightarrow A)$, $(\neg A \rightarrow \neg B) \rightarrow (B \rightarrow A)$, $[A \rightarrow (B \rightarrow C)] \rightarrow [(A \rightarrow B) \rightarrow (A \rightarrow C)]$, etc.
 - ▶ und **Modus Ponens**: wenn man bereits Formeln A und $(A \rightarrow B)$ abgeleitet hat, dann kann man daraus B ableiten.
- **Semi-Entscheidungsverfahren**: einfach alle möglichen Ableitungen in einem vollständigen Kalkül aufzählen.
 - ▶ Wenn die Formel ϕ gültig ist, wird man sie irgendwann ableiten.
 - ▶ Bei ungültigem ϕ werden endlos neue Formeln abgeleitet.

Teil 1: Einführung in Berechenbarkeit und Entscheidbarkeit

Teil 1.1: Probleme, Programme, Algorithmen

Teil 1.2: Berechenbarkeit, Entscheidbarkeit

Teil 1.3: Semi-Entscheidbarkeit

Teil 1.4: Komplement

Teil 1.5: Jenseits der Semi-Entscheidbarkeit

Teil 1.6: Reduktionen

Komplement eines Entscheidungsproblems

- Sei $\mathcal{P}$ ein Entscheidungsproblem, d.h.: eine unendliche Menge von möglichen Instanzen plus ja/nein-Frage.
- Das **Komplement** von $\mathcal{P}$ (oder “**co-Problem** von $\mathcal{P}$ ”) besteht aus der selben Menge von Instanzen wie $\mathcal{P}$ aber mit verneinter Frage.
- Das co-Problem von $\mathcal{P}$ wird üblicherweise als $\text{co-}\mathcal{P}$ bezeichnet.
- Das co-Problem von $\text{co-}\mathcal{P}$ ist $\mathcal{P}$ (wegen doppelter Verneinung).

Wiederholung: GRAPH-ERREICHBARKEIT

INSTANZ: Ein Graph $G = (V, E)$ und Knoten $u, v \in V$.

FRAGE: Gibt es im Graph G einen Pfad von u nach v ?

Das Komplement von GRAPH-ERREICHBARKEIT ist folgendes Problem:

NICHT-ERREICHBARKEIT (oder CO-ERREICHBARKEIT)

INSTANZ: Ein Graph $G = (V, E)$ und Knoten $u, v \in V$.

FRAGE: Gibt es im Graph G **keinen** Pfad von u nach v ?

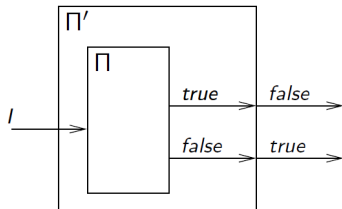
Eigenschaften der Komplementierung

Theorem

Wenn ein Problem $\mathcal{P}$ entscheidbar ist, dann ist auch sein Komplement $\text{co-}\mathcal{P}$ entscheidbar

Beweis:

- Wenn $\mathcal{P}$ entscheidbar ist, dann gibt es ein Programm Π , das für alle positiven Instanzen von $\mathcal{P}$ *true* und für alle negativen Instanzen von $\mathcal{P}$ *false* zurückgibt.
- Wir können Π zu einem neuen Programm Π' ändern, indem wir einfach den Ausgabewert invertieren.
- Dann ist Π' eine Entscheidungsprozedur für $\text{co-}\mathcal{P}$.



Boolean Π' (String I)

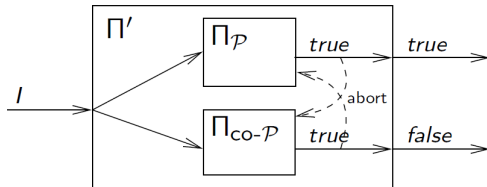
```
if  $\Pi(I) = \text{true}$   
  then return false  
  else return true;
```

Theorem

Sei $\mathcal{P}$ ein Entscheidungsproblem mit Komplement $\text{co-}\mathcal{P}$. Wenn sowohl $\mathcal{P}$ als auch $\text{co-}\mathcal{P}$ *semi-entscheidbar* ist, dann ist $\mathcal{P}$ *entscheidbar*.

Beweis:

- Da $\mathcal{P}$ semi-entscheidbar ist, gibt es ein Programm $\Pi_{\mathcal{P}}$, das auf den positiven Instanzen von $\mathcal{P}$ terminiert und *true* ausgibt.
- Da $\text{co-}\mathcal{P}$ semi-entscheidbar ist, gibt es ein Programm $\Pi_{\text{co-}\mathcal{P}}$, das auf den positiven Instanzen von $\text{co-}\mathcal{P}$ (d.h.: auf den negativen Instanzen von $\mathcal{P}$) terminiert und *true* ausgibt.
- Wir definieren ein Programm Π' , das $\Pi_{\mathcal{P}}$ und $\Pi_{\text{co-}\mathcal{P}}$ parallel ausführt. Eines der beiden Programme terminiert garantiert. Dann liefert Π' den entsprechenden Output und stoppt das andere Programm.



Teil 1: Einführung in Berechenbarkeit und Entscheidbarkeit

Teil 1.1: Probleme, Programme, Algorithmen

Teil 1.2: Berechenbarkeit, Entscheidbarkeit

Teil 1.3: Semi-Entscheidbarkeit

Teil 1.4: Komplement

Teil 1.5: Jenseits der Semi-Entscheidbarkeit

Teil 1.6: Reduktionen

Jenseits der Semi-Entscheidbarkeit

- Wir haben Probleme gesehen, die unentscheidbar aber semi-entscheidbar sind.
- Frage: Gibt es Probleme, die nicht einmal semi-entscheidbar sind?

Theorem

Sei $\mathcal{P}$ ein Entscheidungsproblem, so dass (1) $\mathcal{P}$ unentscheidbar ist und (2) $\mathcal{P}$ semi-entscheidbar ist. Dann ist $\text{co-}\mathcal{P}$ nicht semi-entscheidbar.

Beweis.

Indirekt: Angenommen, $\text{co-}\mathcal{P}$ ist semi-entscheidbar. Das heißt, dass sowohl $\mathcal{P}$ als auch $\text{co-}\mathcal{P}$ semi-entscheidbar ist. Dann folgt aus dem vorigen Theorem, dass $\mathcal{P}$ entscheidbar ist. Widerspruch zur Annahme (1). $\square$

Aus dem Theorem folgt, dass die co-Probleme von HALTEPROBLEM, KORREKTHEIT, ERREICHBARER-CODE und ENTSCHEIDUNGSPROBLEM nicht semi-entscheidbar sind.

Jenseits der Semi-Entscheidbarkeit

- Bis jetzt haben wir nur Probleme $\mathcal{P}$ gesehen, für die zumindest eines der Probleme $\mathcal{P}$ und $\text{co-}\mathcal{P}$ semi-entscheidbar ist.
- **Frage:** Gibt es Probleme, so dass weder $\mathcal{P}$ noch $\text{co-}\mathcal{P}$ semi-entscheidbar ist?
- **Antwort:** Ja, solche Probleme gibt es. Wir werden 3 Beispiele von Problemen sehen, bei denen man sowohl für positive als auch für negative Instanzen implizit das co-HALTEPROBLEM lösen müsste.

Beispiele für Nicht-Semi-Entscheidbarkeit (1)

SELBER-OUTPUT

INSTANZ: Programme Π_1, Π_2 und Input String I .

FRAGE: Haben Π_1 und Π_2 auf Input I das gleiche Verhalten?

Das heißt: entweder terminieren Π_1 und Π_2 bei Input I und liefern das gleiche Ergebnis oder beide Programme terminieren nicht bei Input I ?

Intuition:

- Wir können positive Instanzen, bei denen keines der Programme Π_1 und Π_2 auf Input I terminiert, nicht erkennen. Denn wenn wir das könnten, wäre Nicht-Termination (sprich: das co-HALTEPROBLEM) semi-entscheidbar. Ein Widerspruch!
- Wir können aber auch negative Instanzen nicht erkennen, bei denen eines der Programme Π_1 und Π_2 auf I terminiert und das andere nicht. Denn wenn wir das könnten, würde das wiederum bedeuten, dass Nicht-Termination (sprich: das co-HALTEPROBLEM) semi-entscheidbar wäre. Ein Widerspruch!

Beispiele für Nicht-Semi-Entscheidbarkeit (2)

PROGRAMM-ÄQUIVALENZ

INSTANZ: 2 Programme Π_1, Π_2 (die beide als Input einen String über einem vorgegebenen Alphabet Σ nehmen).

FRAGE: Sind Π_1 and Π_2 **äquivalent**?

D.h.: gilt für jeden Input $I \in \Sigma^*$, dass entweder Π_1 und Π_2 terminieren und das gleiche Ergebnis liefern oder beide Programme endlos laufen?

Intuition:

- Das Problem SELBER-OUTPUT vergleicht das Verhalten von 2 Programmen auf *einem* Input I , wohingegen das Problem PROGRAMM-ÄQUIVALENZ das Verhalten von 2 Programmen auf *unendlich vielen* Inputs I vergleicht.
- Wenn also schon das “einfachere” Problem und sein co-Problem nicht semi-entscheidbar sind, dann auch das “schwierigere” Problem.

Beispiele für Nicht-Semi-Entscheidbarkeit (3)

UNIVERSELLES-HALTEPROBLEM

INSTANZ: Programm Π (das als Input einen String über einem vorgegebenen Alphabet Σ nimmt).

FRAGE: Hält Π auf jedem beliebigen Input String $I \in \Sigma^*$?

Intuition:

- Wir können positive Instanzen nicht erkennen, weil wir für unendlich viele Instanzen $I \in \Sigma^*$ überprüfen müssten, ob Π auf Input I hält.
- Wir können negative Instanzen nicht erkennen, weil wir dafür die Nicht-Termination von Π auf irgendeinem String I erkennen müssten. Das entspricht aber dem co-HALTEPROBLEM.

Bemerkung: Wir haben für die Nicht-Semi-Entscheidbarkeit bis jetzt nur die Intuition beschrieben. Für einen formalen Beweis benötigen wir den Begriff der **Reduktion** (= das wichtigste Werkzeug, um die Schwierigkeit beim Lösen von 2 Problemen zu vergleichen!).

Teil 1: Einführung in Berechenbarkeit und Entscheidbarkeit

Teil 1.1: Probleme, Programme, Algorithmen

Teil 1.2: Berechenbarkeit, Entscheidbarkeit

Teil 1.3: Semi-Entscheidbarkeit

Teil 1.4: Komplement

Teil 1.5: Jenseits der Semi-Entscheidbarkeit

Teil 1.6: Reduktionen

Erste Idee von Reduktionen

Idee (nicht auf die theoretische Informatik beschränkt)

- Angenommen, wir wollen ein **neues Problem A** lösen, (d.h.: wir wollen einen Algorithmus für das Problem A entwickeln).
- Und angenommen, **wir wissen**, wie man ein verwandtes **Problem B** löst (d.h.: wir haben bereits einen passenden Algorithmus für das Problem B).
- **Idee.** Wir könnten versuchen, A zu lösen, indem wir es zum Problem B umformen, (d.h.: wir entwickeln einen Algorithmus für Problem A , der den Algorithmus für Problem B verwendet).

Schlussfolgerung: Wenn diese Strategie funktioniert, sagen wir:
“**Problem A wird auf Problem B reduziert**” und wir schreiben **$A \leq B$** .
 $\implies$ Problem **A ist mindestens so leicht** lösbar wie Problem B .

Zweite Idee von Reduktionen

Idee (typisch für Berechenbarkeit und Komplexität)

- Angenommen, es gelingt uns nicht, für ein **neues Problem B** ein geeignetes Lösungsverfahren zu finden, d.h.: wir finden keinen (effizienten) Algorithmus oder nicht einmal ein Semi-Entscheidungsverfahren für Problem B .
- Und angenommen, **wir wissen**, dass ein verwandtes **Problem A schwer zu lösen** ist, d.h.: es wurde bereits bewiesen, dass es keinen (effizienten) Algorithmus oder nicht einmal ein Semi-Entscheidungsverfahren für Problem A gibt.
- **Idee.** Wir entwickeln einen Algorithmus für Problem A , der einen (hypothetischen) Algorithmus für Problem B verwendet.

Schlussfolgerung: Wenn diese Strategie funktioniert, sagen wir wiederum: **“Problem A wird auf Problem B reduziert”** und wir schreiben **$A \leq B$** .
 $\implies$ Problem **B ist mindestens so schwer** lösbar wie Problem A .

Beschränkung der Ressourcen

Motivation

- Eine Problemreduktion von A nach B sollte einfacher sein als die beiden betrachteten Probleme.
- Wenn das nicht der Fall wäre, könnte die Reduktion einen Teil der Komplexität von A beseitigen und ein Vergleich zwischen (den Schwierigkeitsgraden von) A und B wäre nicht mehr möglich.

Typische Anforderung in der Berechenbarkeitstheorie

Reduktionen müssen **berechenbar** sein.

Typische Anforderung in der Komplexitätstheorie

Reduktionen müssen **effizient** berechenbar sein, z.B. in polynomieller Zeit, d.h.: $O(n^k)$ für Instanzen der Größe n und Konstante $k \geq 1$.

Erste Art von Reduktionen: “Turing” Reduktionen

Idee

- Wir konstruieren ein Lösungsverfahren für Problem A , das ein Lösungsverfahren für Problem B als **Unterprozedur** verwendet.
- Das Lösungsverfahren für Problem A besteht also aus einem **Steuerungsprogramm**, das die Unterprozedur für Instanzen von Problem B “beliebig” oft aufrufen darf.
- **Ressourcenbeschränkung**: Das Steuerungsprogramm selbst muss ein (effizienter) Algorithmus sein. Im Fall einer Beschränkung auf polynomielle Zeit, heißt so eine Reduktion “**Cook Reduktion**”.

Zweite Art von Reduktionen: “Many-One” Reduktionen

Idee

- Definiere eine **Funktion** R von der Menge der Instanzen von Problem A auf die Menge der Instanzen von Problem B , d.h.: jede Instanz x von Problem A wird auf eine Instanz $R(x)$ von Problem B abgebildet.
- Wenn man nun die Instanz $R(x)$ mit einem Lösungsverfahren für Problem B löst, dann ist das Ergebnis für die Instanz $R(x)$ bereits das **korrekte Ergebnis** für die Instanz x des Problems A .
- In diesem Fall sagen wir: **x und $R(x)$ sind äquivalent**.
Für Entscheidungsprobleme A und B bedeutet das: x ist eine positive Instanz von $A \Leftrightarrow R(x)$ ist eine positive Instanz von B .
- **Ressourcenbeschränkung**: Die Funktion R muss (effizient) berechenbar sein. Im Fall einer Beschränkung auf polynomielle Zeit, heißt so eine Reduktion “**Karp Reduktion**”.
- **Bemerkung**: Many-One Reduktionen sind ein Spezialfall von Turing Reduktionen: die Unterprozedur für Problem B wird exakt einmal aufgerufen und die Berechnung endet unmittelbar danach.

Einige klassische Entscheidungsprobleme der Aussagenlogik

SAT (SATISFIABILITY)

INSTANZ: Aussagenlogische Formel ϕ .

FRAGE: Ist ϕ erfüllbar? (d.h.: gibt es eine Wahrheitsbelegung I für die aussagenlogischen Variablen in ϕ , sodass ϕ von I erfüllt wird).

3-SAT

INSTANZ: Aussagenlogische Formel ϕ in 3-CNF (d.h.: konjunktive Normalform, bei der jede Klausel aus höchstens 3 Literalen besteht).

FRAGE: Ist ϕ erfüllbar?

2-SAT

INSTANZ: Aussagenlogische Formel ϕ in 2-CNF (d.h.: konjunktive Normalform, bei der jede Klausel aus höchstens 2 Literalen besteht).

FRAGE: Ist ϕ erfüllbar?

Reduktion von 2-SAT auf GRAPH-ERREICHBARKEIT

Theorem

Das 2-SAT Problem lässt sich mittels Turing Reduktion auf das GRAPH-ERREICHBARKEIT Problem reduzieren.

Bemerkung:

- Das GRAPH-ERREICHBARKEIT Problem ist ein bekanntes, effizient lösbares Problem (sogar in linearer Zeit).
- Wir haben hier also den Fall $A \leq B$, bei dem es für Problem B (= GRAPH-ERREICHBARKEIT) ein effizientes Lösungsverfahren gibt.
- Die Reduktion, die wir hier betrachten, lässt sich jedenfalls in polynomieller Zeit realisieren.
- Schlussfolgerung: Wir bekommen damit ein Lösungsverfahren für Problem A (= 2-SAT), das in polynomieller Zeit läuft.

Reduktion von 2-SAT auf GRAPH-ERREICHBARKEIT

Notation: Für ein Literal α (d.h., aussagenlogische Variable x_i oder ihre Verneinung $\neg x_i$) bezeichnet $\bar{\alpha}$ das *duale* Literal, d.h.: wenn $\alpha = x_i$ ist, dann ist $\bar{\alpha} = \neg x_i$; und wenn $\alpha = \neg x_i$ ist, dann ist $\bar{\alpha} = x_i$.

Von der Formel zum Graphen

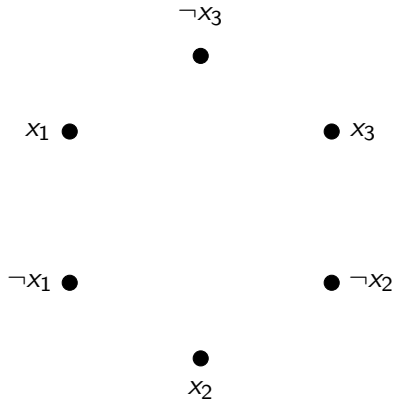
Sei ϕ eine beliebige Instanz von 2-SAT mit Variablen $X = \{x_1, \dots, x_n\}$. Wir definieren daraus folgenden Graphen $G_\phi = (V, E)$:

- Wir setzen $V = \{x_1, \dots, x_n, \neg x_1, \dots, \neg x_n\}$, d.h.:
Die Knotenmenge V von G_ϕ enthält die Literale, die man mit den aussagenlogischen Variablen in X bilden kann.
- Die Kantenmenge E von G_ϕ enthält alle Kanten der Form $(\bar{\alpha}, \beta)$ und $(\bar{\beta}, \alpha)$, für die die Formel ϕ eine Klausel $\alpha \vee \beta$ enthält.

Intuition dieser Konstruktion: Eine Klausel der Form $\alpha \vee \beta$ ist logisch äquivalent zu den Implikationen $\bar{\alpha} \rightarrow \beta$ und $\bar{\beta} \rightarrow \alpha$. Auf diese Weise wollen wir logische Implikation mittels Pfaden im Graph codieren.

Example

$$\phi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$$

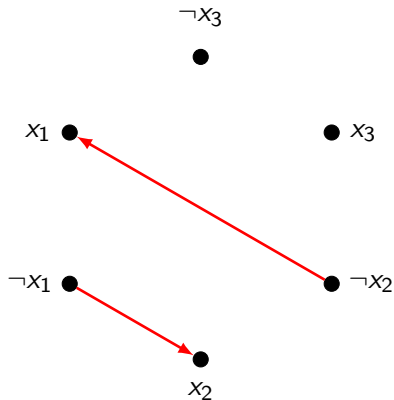


Example

$$\phi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$$

$$\neg x_1 \rightarrow x_2$$

$$\neg x_2 \rightarrow x_1$$

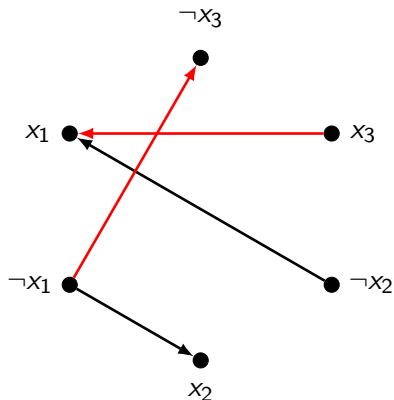


Example

$$\phi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$$

$$\neg x_1 \rightarrow \neg x_3$$

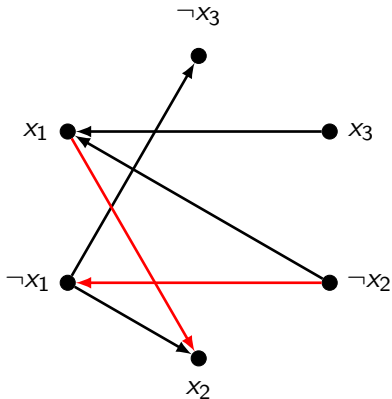
$$x_3 \rightarrow x_1$$



Example

$$\phi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$$

$$\begin{array}{lcl} x_1 & \rightarrow & x_2 \\ \neg x_2 & \rightarrow & \neg x_1 \end{array}$$

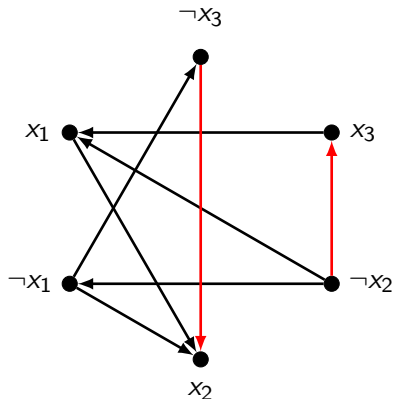


Example

$$\phi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$$

$$\neg x_2 \rightarrow x_3$$

$$\neg x_3 \rightarrow x_2$$



Zusammenhang zwischen ϕ und G_ϕ

Beobachtung (ohne Beweis)

Seien α, β zwei beliebige Literale über den Variablen X in ϕ (und somit auch zwei Knoten in G_ϕ). Dann sind folgende Behauptungen äquivalent:

- (1) In jedem Modell I von ϕ ist $\alpha \rightarrow \beta$ true. (geschrieben: $\phi \models (\alpha \rightarrow \beta)$)
- (2) Im Graph G_ϕ gibt es einen Pfad von α nach β .

Problematische Pfade im Graph G_ϕ

- **Frage:** Kann ϕ ein Modell haben, wenn es in G_ϕ einen Pfad von x_i nach $\neg x_i$ gibt? (d.h.: es gilt $\phi \models (x_i \rightarrow \neg x_i)$?)
Antwort: Ja. Aber in jedem Modell von ϕ muss $I(x_i) = \text{false}$ gelten.
- **Frage:** Kann ϕ ein Modell haben, wenn es in G_ϕ einen Pfad von $\neg x_i$ nach x_i gibt? (d.h.: es gilt $\phi \models (\neg x_i \rightarrow x_i)$?)
Antwort: Ja. Aber in jedem Modell von ϕ muss $I(x_i) = \text{true}$ gelten.
- **Frage:** Und wenn es sowohl einen Pfad von x_i nach $\neg x_i$ als auch von $\neg x_i$ nach x_i in G_ϕ gibt?

Beobachtung (ohne Beweis)

Folgende Behauptungen sind äquivalent:

- (1) Die Formel ϕ ist erfüllbar.
- (2) Es gibt keine Variable $x_i \in X$, für die der Graph G_ϕ sowohl einen Pfad von x_i nach $\neg x_i$ als auch von $\neg x_i$ nach x_i enthält.

Beweisidee: Die Richtung (1) $\Rightarrow$ (2) ist klar nach den Überlegungen auf der vorigen Folie. Die Richtung (2) $\Rightarrow$ (1) ist nicht-trivial. Man kann letztlich ein Modell I von ϕ in mehreren Stufen konstruieren:

- Zuerst werden in I alle Variablen x_i mit false bzw. true belegt, für die es einen Pfad von x_i nach $\neg x_i$ bzw. von $\neg x_i$ nach x_i in G_ϕ .
- Dann werden Wahrheitswerte entlang von allen Pfaden propagiert, z.B.: wenn $I(\alpha) = \text{true}$ gilt und es einen Pfad von α nach β gibt, dann muss auch $I(\beta) = \text{true}$ gelten.
- Wenn dann noch Variablen in I unbestimmt sind, kann man für *eine* dieser Variablen einen beliebigen Wahrheitswert festlegen und diesen dann wieder entlang von Pfaden propagieren.

Damit erhalten wir folgende Turing Reduktion von 2-SAT nach GRAPH-ERREICHBARKEIT:

procedure test2SAT:

Input: propositional formula ϕ in 2-CNF;

Output: true if ϕ is satisfiable and false otherwise.

from ϕ construct G_ϕ ;

for all variables x in ϕ **do** {

if G_ϕ contains a path from x to $\neg x$

 and a path from $\neg x$ to x **then return** false;}

return true;

Bemerkung:

- Beim Test in der if-Anweisung wird jeweils zwei Mal ein Lösungsverfahren für GRAPH-ERREICHBARKEIT als Unterprozedur aufgerufen, und zwar mit den Instanzen $(G_\phi, x, \neg x)$ und $(G_\phi, \neg x, x)$.
- **Korrektheit der Reduktion** = Korrektheit der Prozedur test2SAT. (diese folgt aus den vorigen Beobachtungen und Überlegungen).

Ein klassisches Entscheidungsproblem der Graphentheorie

K-FÄRBBARKEIT

Sei $k \in \mathbb{N}$ mit $k \geq 2$. Dann ist K-FÄRBBARKEIT folgendermaßen definiert:

INSTANZ: ungerichteter Graph $G = (V, E)$

FRAGE: Ist der Graph G k -färbbar?

d.h.: gibt es eine Funktion $f: V \rightarrow \{0, \dots, k-1\}$, so dass für alle Kanten $[v_i, v_j] \in E$ gilt: $f(v_i) \neq f(v_j)$.

Bemerkung:

- Man kann k -Färbbarkeit für beliebige Werte $k \geq 2$ betrachten.
- Im Komplexitätsteil der Vorlesung wird gezeigt werden, dass 3-Färbbarkeit (eig. k -Färbbarkeit für jedes $k \geq 3$) ein schweres Problem ist (d.h. es ist kein effizienter Algorithmus bekannt)
- Wir betrachten hier 2-Färbbarkeit (ein sehr leichtes Problem).

Reduktion von 2-FÄRBBARKEIT auf 2-SAT

Theorem

Das 2-FÄRBBARKEIT Problem lässt sich mittels Many-One Reduktion auf das 2-SAT Problem reduzieren.

Problemreduktion

Sei $G = (V, E)$ eine beliebige Instanz von 2-FÄRBBARKEIT, d.h.:
 $G = (V, E)$ ist ein ungerichteter Graph mit Knotenmenge
 $V = \{v_1, \dots, v_n\}$ und Kantenmenge E .

Daraus definieren wir folgende Instanz φ_G von 2-SAT mit den aussagenlogischen Variablen $x_1, \dots, x_n$:

$$\varphi_G = \bigwedge_{[v_i, v_j] \in E} \left((x_i \vee x_j) \wedge (\neg x_i \vee \neg x_j) \right)$$

Bemerkung:

- Wir haben gerade gezeigt, dass das 2-SAT Problem effizient lösbar ist.
- Aufgrund der Reduktion von 2-FÄRBBARKEIT auf 2-SAT ist natürlich auch 2-FÄRBBARKEIT effizient lösbar (wobei das im Fall von 2-FÄRBBARKEIT auch ohne diese Reduktion leicht zu sehen wäre).
- Reduzierbarkeit ist **transitiv**: Man könnte nun die Reduktion von 2-FÄRBBARKEIT auf 2-SAT und die Reduktion von 2-SAT auf GRAPH-ERREICHBARKEIT **hintereinander ausführen**, und würde damit eine Reduktion von 2-FÄRBBARKEIT auf GRAPH-ERREICHBARKEIT bekommen.

Korrektheit der Reduktion

Wir müssen folgende Äquivalenz zeigen:

$G = (V, E)$ ist eine positive Instanz von 2-FÄRBBARKEIT $\Leftrightarrow$
 φ_G ist eine positive Instanz von 2-SAT.

Üblicherweise geht man so vor, dass man die beiden Implikationen “ $\Rightarrow$ ” und “ $\Leftarrow$ ” getrennt zeigt.

Wir zeigen zuerst die Implikation “ $\Rightarrow$ ”, d.h.:

$G = (V, E)$ ist eine positive Instanz von 2-FÄRBBARKEIT $\Rightarrow$
 φ_G ist eine positive Instanz von 2-SAT.

Und wir zeigen dann die Implikation “ $\Leftarrow$ ”, d.h.:

$G = (V, E)$ ist eine positive Instanz von 2-FÄRBBARKEIT $\Leftarrow$
 φ_G ist eine positive Instanz von 2-SAT.

Korrektheit der Reduktion: Beweis der “ $\Rightarrow$ ”-Richtung

Angenommen $G = (V, E)$ ist eine positive Instanz von 2-FÄRBBARKEIT. Dann gibt es eine Funktion $f: V \rightarrow \{0, 1\}$, die eine gültige 2-Färbung für den Graph $G = (V, E)$ ist, d.h.: für jede Kante $[v_i, v_j]$ in G gilt entweder $(f(v_i) = 0 \text{ und } f(v_j) = 1)$ oder $(f(v_i) = 1 \text{ und } f(v_j) = 0)$.

Wir definieren daraus folgende Wahrheitsbelegung I auf den aussagenlogischen Variablen $x_1, \dots, x_n$:

$$I(x_i) = \begin{cases} \text{false} & \text{falls } f(v_i) = 0 \\ \text{true} & \text{falls } f(v_i) = 1 \end{cases}$$

Wir müssen noch zeigen, dass I die Formel ϕ_G erfüllt. Die Formel ϕ_G ist eine Konjunktion von Teilformeln der Form $(x_i \vee x_j) \wedge (\neg x_i \vee \neg x_j)$. Es reicht daher zu zeigen, dass I jede Klausel $(x_i \vee x_j)$ und $(\neg x_i \vee \neg x_j)$ erfüllt.

Beweis, dass I jede Klausel $(x_i \vee x_j)$ und $(\neg x_i \vee \neg x_j)$ erfüllt:

- (1) Beliebige Klausel der Form $(x_i \vee x_j)$. Laut Problemreduktion enthält ϕ_G so eine Klausel nur dann, wenn es eine Kante $[v_i, v_j]$ in G gibt.

Laut Annahme ist f eine gültige Zweifärbung von G . Also ist mindestens einer der Funktionswerte $f(v_i)$ und $f(v_j)$ gleich 1.

Laut Definition von I gilt dann für mindestens eine der Variablen x_i und x_j , dass sie in I den Wahrheitswert true hat. Somit erfüllt I die Klausel $(x_i \vee x_j)$.

- (2) Beliebige Klausel der Form $(\neg x_i \vee \neg x_j)$. Laut Problemreduktion enthält ϕ_G so eine Klausel nur dann, wenn es eine Kante $[v_i, v_j]$ in G gibt. Laut Annahme ist f eine gültige Zweifärbung von G . Also ist mindestens einer der Funktionswerte $f(v_i)$ und $f(v_j)$ gleich 0.

Laut Definition von I gilt dann für mindestens eine der Variablen x_i und x_j , dass sie in I den Wahrheitswert false hat. Dann hat aber mindestens eines der Literale $\neg x_i$ und $\neg x_j$ den Wahrheitswert true. Somit erfüllt I die Klausel $(\neg x_i \vee \neg x_j)$.

Korrektheit der Reduktion: Beweis der “ $\Leftarrow$ ”-Richtung

Angenommen φ_G ist eine **positive Instanz** von 2-SAT. Dann gibt es eine Wahrheitsbelegung I auf den Variablen $x_1, \dots, x_n$, die die Formel ϕ_G erfüllt. Die Formel ϕ_G ist eine Konjunktion von Teilformeln der Form $(x_i \vee x_j) \wedge (\neg x_i \vee \neg x_j)$. Wenn I die Formel ϕ_G erfüllt, dann erfüllt I also auch jede Teilformel $(x_i \vee x_j) \wedge (\neg x_i \vee \neg x_j)$ von ϕ_G .

Mit Hilfe von I **definieren wir** nun folgende Funktion $f: V \rightarrow \{0, 1\}$:

$$f(v_i) = \begin{cases} 0 & \text{falls } I(x_i) = \text{false} \\ 1 & \text{falls } I(x_i) = \text{true} \end{cases}$$

Wir müssen noch zeigen, dass f eine gültige 2-Färbung von G ist, d.h.: für jede Kante $[v_i, v_j]$ im Graphen G , gilt $f(v_i) \neq f(v_j)$.

Sei also $[v_i, v_j]$ eine *beliebige* Kante im Graphen G . **Laut Problemreduktion** enthält dann ϕ_G die Teilformel $(x_i \vee x_j) \wedge (\neg x_i \vee \neg x_j)$.

Laut Annahme erfüllt die Wahrheitsbelegung I die Formel ϕ_G und somit auch $(x_i \vee x_j) \wedge (\neg x_i \vee \neg x_j)$, d.h.: sowohl $(x_i \vee x_j)$ als auch $(\neg x_i \vee \neg x_j)$ wird von I erfüllt. Daraus folgt:

- Da $(x_i \vee x_j)$ von I erfüllt wird, muss mindestens eine der aussagenlogischen Variablen x_i, x_j in I den Wert *true* haben.
- Da auch $(\neg x_i \vee \neg x_j)$ von I erfüllt wird, muss mindestens eines der Literale $\neg x_i, \neg x_j$ in I *true* sein und somit muss mindestens eine der Variablen x_i, x_j in I den Wert *false* haben.

Insgesamt gilt also, dass eine der aussagenlogischen Variablen x_i, x_j in I den Wert *true* und eine den Wert *false* hat.

Aufgrund der Definition von f ist also einer der Funktionswerte $f(v_i), f(v_j)$ gleich 1 und einer der Funktionswerte $f(v_i), f(v_j)$ gleich 0. Also insbesondere gilt $f(v_i) \neq f(v_j)$. Da wir diese Eigenschaft für eine beliebige Kante $[v_i, v_j]$ in G gezeigt haben, gilt die Eigenschaft für *alle* Kanten in G , d.h.: f ist eine gültige 2-Färbung von G .

Typische Vorgangsweise bei Korrektheitsbeweisen

- Zur Erinnerung: eine Many-One Reduktion R von einem Entscheidungsproblem A auf ein Entscheidungsproblem B ist korrekt, wenn für jede beliebige Instanz x von A die Äquivalenz gilt: x ist eine positive Instanz von $A \Leftrightarrow R(x)$ ist eine positive Instanz von B .
- Üblicherweise zeigt man die 2 Richtungen " $\Rightarrow$ " und " $\Leftarrow$ " der Äquivalenz getrennt.
- Der Beweis beginnt immer mit der Annahme "Sei x eine positive Instanz von A " bzw. "Sei $R(x)$ eine positive Instanz von B ".
- Mittels *schlüssiger* Beweisschritte versucht man zu zeigen, dass man dann auch beim anderen Problem eine positive Instanz hat.
- Typische Beweisschritte sind, wenn etwas *aufgrund der Annahme*, *aufgrund einer Definition* (z.B. einer "Lösung") oder *aufgrund der Problemreduktion* gilt.
- Es erhöht die Lesbarkeit eines Beweises, wenn man im Beweis klarstellt, was man gerade zeigt bzw. was noch zu zeigen ist.

Unentscheidbarkeitsbeweise mittels Reduktionen

- Wir wissen, dass das HALTEPROBLEM unentscheidbar ist.
- Außerdem haben wir gezeigt, dass es noch viele weitere (sogar überabzählbar viele) unentscheidbare Probleme gibt.
- Um die Unentscheidbarkeit von einem weiteren Problem $\mathcal{P}$ zu beweisen, könnten wir einen **ähnlich komplizierten Beweis wie das Diagonalargument** für das HALTEPROBLEM suchen.
- **Bessere Idee.** Um die Unentscheidbarkeit von einem (neuen) Problem $\mathcal{P}$ zu beweisen, können wir **das HALTEPROBLEM** (oder irgendein anderes, als unentscheidbar bewiesenes Problem) **auf $\mathcal{P}$ reduzieren**.
- **Begründung:** Angenommen es gäbe eine Entscheidungsprozedur $\Pi_{\mathcal{P}}$ für das Problem $\mathcal{P}$. Dann konstruieren wir eine Entscheidungsprozedur Π_h für das HALTEPROBLEM wie folgt:
 - ▶ Π_h nimmt als Input eine Instanz $x = (\Pi, I)$ des HALTEPROBLEMS;
 - ▶ Π_h wendet die Problemreduktion R vom HALTEPROBLEM auf $\mathcal{P}$ auf diese Instanz x an, d.h.: es wird $R(x)$ berechnet;
 - ▶ mit $R(x)$ als Input wird $\Pi_{\mathcal{P}}$ aufgerufen;
 - ▶ der Output von $\Pi_{\mathcal{P}}$ ist auch der Output von Π_h .

Beispiele von unentscheidbaren Problemen

Wiederholung: KORREKTHEIT

INSTANZ: (Quellcode von einem) Programm Π und 2 Strings I_1, I_2 .

FRAGE: Liefert Π bei Aufruf mit Input I_1 als Output den String I_2 ?

Theorem

Das KORREKTHEIT Problem ist unentscheidbar.

Beweis mittels Reduktion vom HALTEPROBLEM

Sei (Π, I) eine beliebige Instanz des HALTEPROBLEMS, d.h.: Π ist ein SIMPLE Programm und I ist ein möglicher Input-String für Π .

Daraus definieren wir eine Instanz (Π', I_1, I_2) des KORREKTHEIT Problems, indem wir $I_1 = I_2 = I$ setzen und Π' wie folgt definieren:

```
String  $\Pi'$  (String  $S$ ) {  
    call  $\Pi(S)$ ;  
    return  $S$ ; }
```

Fortsetzung des Unentscheidbarkeitsbeweises

Wir müssen noch die **Korrektheit der Reduktion** zeigen, d.h.: wir müssen zeigen, dass die ursprüngliche Instanz des HALTEPROBLEMS und die (in der Problemreduktion) definierte Instanz des KORREKTHEIT Problems **äquivalent** sind, d.h.:

(Π, I) ist eine positive Instanz des HALTEPROBLEMS $\Leftrightarrow$
 (Π', I_1, I_2) ist eine positive Instanz des KORREKTHEIT Problems.

“ $\Rightarrow$ ” Angenommen (Π, I) ist eine positive Instanz des HALTEPROBLEMS, d.h.: Π hält bei Input I . Laut Konstruktion von Π' , hält dann auch Π' auf dem Input $I_1 = I$ und liefert als Output den String $I_2 = I$, d.h.: (Π', I_1, I_2) ist eine positive Instanz des KORREKTHEIT Problems.

“ $\Leftarrow$ ” Angenommen (Π', I_1, I_2) ist eine positive Instanz des KORREKTHEIT Problems, d.h.: Π' hält bei Input I_1 und liefert I_2 als Output. Laut Problemreduktion gilt $I_1 = I_2 = I$ und der Aufruf von Π' mit Input I führt zu einem Aufruf von Π mit Input I . Da Π' auf diesem Input hält (und I_2 liefert), muss auch Π auf dem Input I halten, d.h.: (Π, I) ist eine positive Instanz des HALTEPROBLEMS.

Beispiele von unentscheidbaren Problemen (Forts.)

Wiederholung: SELBER-OUTPUT

INSTANZ: Programme Π_1, Π_2 und Input String I .

FRAGE: Haben Π_1 und Π_2 auf Input I das gleiche Verhalten?

Das heißt: entweder terminieren Π_1 und Π_2 bei Input I und liefern das gleiche Ergebnis oder beide Programme terminieren nicht bei Input I ?

Theorem

Das SELBER-OUTPUT Problem ist unentscheidbar.

Beweis mittels Reduktion vom HALTEPROBLEM

Sei (Π, I) eine beliebige Instanz des HALTEPROBLEMS, d.h.: Π ist ein SIMPLE Programm und I ist ein möglicher Input-String für Π .

Daraus definieren wir eine Instanz (Π_1, Π_2, I') des SELBER-OUTPUT Problems mit $I' = I$ und folgenden Programmen Π_1, Π_2 :

```
String  $\Pi_1$  (String  $S$ ) {  
  return  $S$ ; }
```

```
String  $\Pi_2$  (String  $S$ ) {  
  call  $\Pi(S)$ ; return  $S$ ; }
```

Intuition der Reduktion:

- Programm Π_1 liefert bei Input I *unmittelbar* den Output I .
Offensichtlich hält Programm Π_1 garantiert.
- Programm Π_2 ruft zunächst Π mit Input I auf und gibt dann den Output I aus – vorausgesetzt, dass Π bei Input I hält.

Fortsetzung des Unentscheidbarkeitsbeweises

Wir müssen noch die **Korrektheit der Reduktion** zeigen, d.h.:

(Π, I) ist eine positive Instanz des HALTEPROBLEMS $\Leftrightarrow$

(Π_1, Π_2, I) ist eine positive Instanz des SELBER-OUTPUT Problems.

Wir zeigen sofort **beide Richtungen der Äquivalenz**:

(Π, I) ist positive Instanz des HALTEPROBLEMS, d.h.: Π hält bei Input I .

$\Leftrightarrow$ Programm Π_2 terminiert bei Input I und liefert den Output I . $\Leftrightarrow \Pi_1$ und Π_2 haben auf Input I das gleiche Verhalten. (Da Programm Π_1 immer terminiert und bei Input I den Output I liefert.) $\Leftrightarrow (\Pi_1, \Pi_2, I)$ ist eine positive Instanz des SELBER-OUTPUT Problems.

Beweise der Nicht-Semi-Entscheidbarkeit

Idee:

- Wir wissen, dass das co-HALTEPROBLEM nicht semi-entscheidbar ist.
- Wir können daher die Nicht-Semi-Entscheidbarkeit eines (neuen) Problems mittels Reduktion vom co-HALTEPROBLEM beweisen.

Beobachtung:

- Angenommen wir können für 2 Probleme zeigen, dass $\mathcal{P}_1 \leq \mathcal{P}_2$ gilt. Dann gilt offensichtlich auch $\text{co-}\mathcal{P}_1 \leq \text{co-}\mathcal{P}_2$.
- **Begründung:** Korrektheit einer Reduktion R bedeutet, dass für jede Instanz x von $\mathcal{P}_1$ gilt: x ist positive Instanz von $\mathcal{P}_1 \Leftrightarrow R(x)$ ist positive Instanz von $\mathcal{P}_2$. Dann gilt natürlich auch: x ist negative Instanz von $\mathcal{P}_1$ (und somit positive Instanz von $\text{co-}\mathcal{P}_1$) $\Leftrightarrow R(x)$ ist negative Instanz von $\mathcal{P}_2$ (und somit positive Instanz von $\text{co-}\mathcal{P}_2$).
- Daher gilt: $\mathcal{P}_1 \leq \mathcal{P}_2 \Leftrightarrow \text{co-}\mathcal{P}_1 \leq \text{co-}\mathcal{P}_2$
- **Schlussfolgerung:** Die Probleme co-KORREKTHEIT und co-SELBER-OUTPUT sind **nicht semi-entscheidbar**.

Weitere nicht-semi-entscheidbare Probleme

Wir haben gerade gesehen, dass das co-SELBER-OUTPUT Problem nicht semi-entscheidbar ist. Es gilt aber auch:

Theorem

Das SELBER-OUTPUT Problem ist nicht semi-entscheidbar.

Beweis mittels Reduktion vom co-HALTEPROBLEM

Sei (Π, I) eine beliebige Instanz des co-HALTEPROBLEMS , d.h.: Π ist ein SIMPLE Programm und I ist ein möglicher Input-String für Π .

Daraus definieren wir eine Instanz (Π_1, Π_2, I') des SELBER-OUTPUT Problems mit $I' = I$ und folgenden Programmen Π_1, Π_2 :

```
String  $\Pi_1$  (String  $S$ ) {  
  while (true) do {} };
```

```
String  $\Pi_2$  (String  $S$ ) {  
  call  $\Pi(S)$ ; return  $S$ ; }
```

Intuition der Reduktion:

- Programm Π_1 ignoriert den Input I und läuft sofort in eine *Endlosschleife*, d.h.: Π_1 terminiert garantiert nicht.
- Programm Π_2 ruft zunächst Π mit Input I auf und gibt dann den Output I aus, d.h.: Π_2 terminiert auf Input I genau dann, wenn Π auf Input I terminiert.

Fortsetzung des Beweises

Wir müssen noch die **Korrektheit der Reduktion** zeigen, d.h.:

(Π, I) ist eine positive Instanz des co-HALTEPROBLEMS $\Leftrightarrow$
 (Π_1, Π_2, I) ist eine positive Instanz des SELBER-OUTPUT Problems.

Wir zeigen sofort **beide Richtungen der Äquivalenz**:

(Π, I) ist eine positive Instanz des co-HALTEPROBLEMS, d.h.:
 Π hält bei Input I nicht. $\Leftrightarrow$ Programm Π_2 terminiert auf Input I nicht. $\Leftrightarrow$
 Π_1 und Π_2 haben auf Input I das gleiche Verhalten. (Da Programm Π_1 bei jedem Input endlos läuft.) $\Leftrightarrow (\Pi_1, \Pi_2, I)$ ist eine positive Instanz des SELBER-OUTPUT Problems.

Zusammenfassung von Teil 1 der LVA

- Formale Definition von “Problemen” (Instanzen, Frage)
- verschiedene Arten von Problemen (insbes.: Entscheidungsprobleme)
- unser Modell für “Berechenbarkeit” (SIMPLE Programmiersprache)
- Berechenbarkeit, Entscheidbarkeit, Semi-Entscheidbarkeit
- Abzählbarkeit (Cantor’sches Abzählprinzip)
- Überabzählbarkeit (Diagonalargument)
- HALTEPROBLEM (noch ein Diagonalargument)
- co-Probleme und ihre Eigenschaften: (Semi-)Entscheidbarkeit
- unentscheidbare / nicht-semi-entscheidbare Probleme (Intuition)
- **Reduktionen** (Motivation, Turing vs. Many-One)
- **Korrektheitsbeweis von Reduktionen**